Week 3: Linear Regression (continued)*

Instructor: Dr. Federico Fusco

November 28, 2025 - version 2

1 Introduction

Last week, we studied the linear regression problem and found a satisfying *closed formula*. We recall the setting. We are given n points $(x_1, y_1), \dots, (x_n, y_n)$, where each $x_i \in \mathcal{X}$, and $y_i \in \mathbb{R}$. On the space $\mathcal{X}$, we are given d feature maps $\phi_i : \mathcal{X} \to \mathbb{R}$, which induce functions of the type $f_\theta : \mathcal{X} \to \mathbb{R}$ for $\theta \in \mathbb{R}^d$ as follows:

$$f_\theta(x) = \sum_{j=1}^{d} \phi_j(x) \cdot \theta_j = \phi(x)^T \theta,$$

where $\phi(x)$ is the d-dimensional vector whose j^{th} entry is $\phi_j(x)$. Note, this problem is called *linear* regression because the function $f_\theta(x)$ is linear in θ (so, possibly, not in x)!

Our optimization problem. Our goal is to find $\theta \in \mathbb{R}^d$ that minimizes the mean squared error on the input points with respect to the feature maps $\phi_1, \dots, \phi_d$. Stated differently, we want to find the vector θ, that minimizes:

$$\hat{R}(\theta) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2 = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \phi(x)^T \theta)^2 = \frac{1}{2n} ||y - \Phi\theta||_2^2.$$

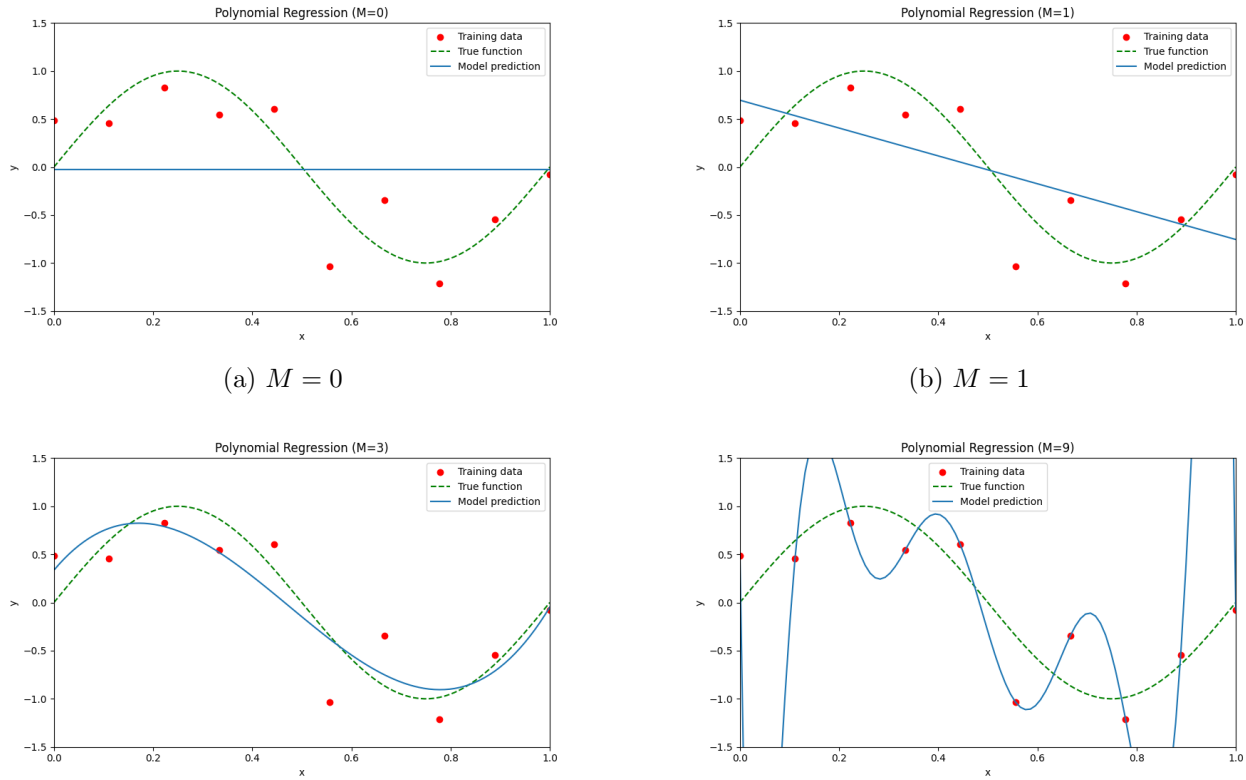
We have observed that the solution of the minimization problem is given by the vector $\hat{\theta} \in \mathbb{R}^d$, under the assumption that $\Phi^T \Phi$ is invertible.

$$\hat{\theta} = (\Phi^T \Phi) \Phi^T y.$$

2 Polynomial Regression and Overfitting

A powerful choice of the feature maps is given by polynomials. In this case, we have $\phi_0(x) = 1$, $\phi_1(x) = x, \dots, \phi_i(x) = x^i$, so that the generic function $f_\theta(x)$ looks like $f_\theta(x) = \sum_{i=0}^{M} \theta_i x^i$. In particular, with $\theta \in \mathbb{R}^{M+1}$, we capture all polynomials of degree up to M. Since we can actually choose M, it is natural to try different ones, as we can see in Figure 1. Note, if we have n points and set $M = n - 1$, then we will suffer zero error, as it is generally possible to find a d-dimensional polynomial that passes through $d - 1$ points. By increasing the degree M, we observe

*These notes are meant as a teaching support for the students of the "Machine Learning" course for the Master's Degree in Engineering in Computer Science and Artificial Intelligence that I am teaching in Fall 2025. They are likely full of typos and imprecisions, so please get in touch with me in class if something feels off. From time to time, I may update new versions of these notes, so make sure to download the latest version.



(a) $M = 0$ (b) $M = 1$

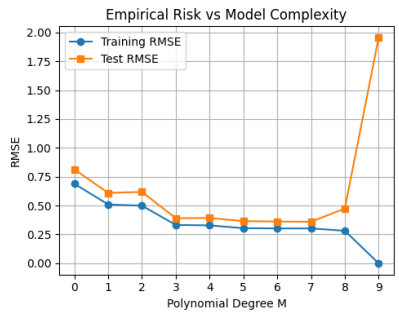
(c) $M = 3$ (d) $M = 9$

Figure 1: Predicted curves for various degrees M

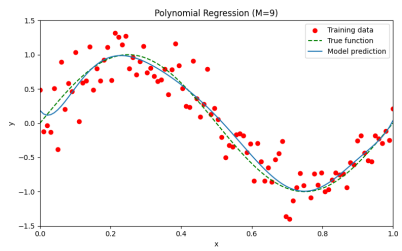
the following phenomenon, which is fairly common in machine learning models: as the complexity of the underlying model increases (in this case, the degree M of the polynomial family considered as predictors), the training error (i.e., the error made on the portion of the data that is actually used to compute $\hat{\theta}$!) decreases. To visualize this, we construct a synthetic dataset, starting from a base function and adding some noise (See Figure 1). This is pictorially represented by the blue line in Figure 2a, where the error clearly drops to zero as $M \geq 9$.

However, we should not forget that our final goal is **not** to minimize the training error, but to make meaningful predictions on future data points. This is called *generalization* and is the core of machine learning. If the complexity of the model increases "too much", then the prediction may get too tailored to the data in the training set, and the predictor may lose generalization power. This is called *overfitting*, and you can visualize it in the orange line of Figure 2a. From that Figure, it seems like the sweet spot for the the complexity of the model, given the (tiny) amount of data we have, is $M = 6$ or 7. There are some standard tools to tackle overfitting. We present here some of them that are well suited for linear regression.

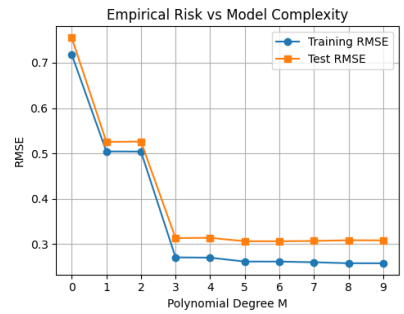
Adding more data point. A good rule of thumb is that *more* data points buy you the possibility of using *more complex* models. If we stick to the polynomial regression example, having $n = 100$ points (as opposed to the $n = 10$), allows us to get a good generalization error while exploiting the additional flexibility of using the polynomials of degree 9. If you compare Figure 2b and Figure 1d, in fact, you see that increasing the training set did help in getting a better approximation of the underlying function that we are trying to estimate. This is reflected by the train vs test error plot



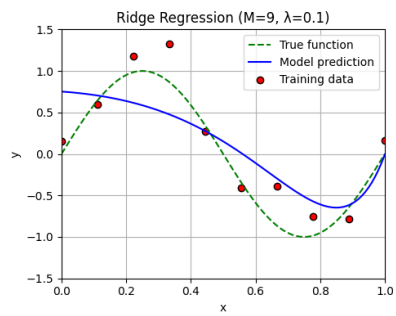
(a) Train vs Test Error for $n = 10$



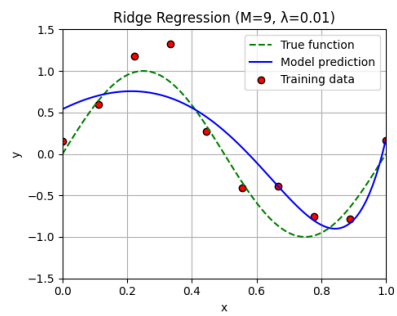
(b) $M = 9$ for $n = 100$



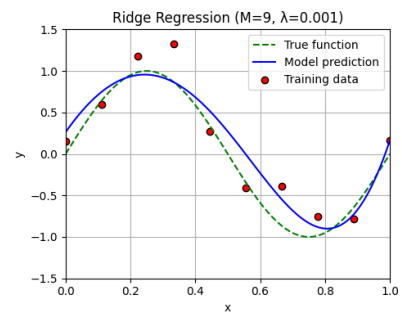
(c) Train vs Test Error for $n = 100$



(d) Ridge with $\lambda = 0.1$



(e) Ridge with $\lambda = 10^{-2}$



(f) Ridge with $\lambda = 10^{-3}$

Figure 2: Overfitting and Methods to avoid it.

reported in Figure 2c.

Adding a Regularizer. Another technique consists in adding a regularization term to the objective that is minimized in the training. Namely, in linear regression, instead of minimizing $\hat{R}(\theta) = \frac{1}{2n}||\Phi\theta + y||_2^2$, we aim at minimizing $\hat{R}(\theta) = \frac{1}{2n}||\Phi\theta + y||_2^2 + \lambda\Psi(\theta)$, for some regularization function Ψ, and parameter λ. There are three natural choices of regularizer:

- Ridge regularizer: $\Psi(\theta) = ||\theta||_2^2$. Useful to force "small" values of θ. It has the desirable feature of being convex and differentiable.
- Lasso regularizer: $\Psi(\theta) = ||\theta||_1$. It may force θ to set to zero some coordinates, which is good as it implicitly performs "feature selections".
- Elastic Regularizer, which is a convex combination of Lasso and Ridge.

Once we decide which regularizer to use, it is still important to set the right hyper parameter λ. You can see the effect of the regularization in Figures 2d to 2f. Note:

3 The Computational Complexity of the OLS Estimator

We now want to understand the computational complexity of computing the closed formula of $\hat{\theta}$, assuming that computing the generic feature map ϕ_j takes constant time. We need to perform the following operations:

- Compute the d feature maps ϕ_i on each one of the n points: $O(nd)$ time steps

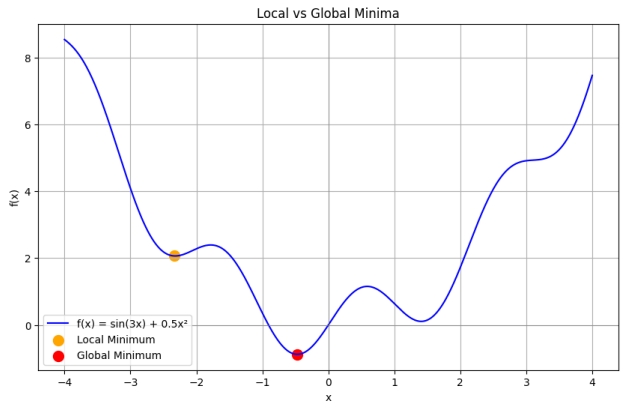


Figure 3: Visualization of the difference between global and local minima.

- Compute each one of the $d \times d$ entries of $\Phi^T\Phi$, each one given by the scalar product of two n-dimensional vectors, for an overall running time of $O(nd^2)$
- Compute the inverse of $\Phi^T\Phi$, which, in general, takes $O(d^3)$ time steps (e.g., with Gaussian Elimination)
- Compute each one of the d entries of $\Phi^T y$, each one given by the scalar product of two n-dimensional vectors, for an overall running time of $O(nd)$
- Compute the product of the $d \times d$ matrix $(\Phi^T\Phi)^{-1}$, and the d dimensional vector $\Phi^T y$, this takes $O(d^2)$ time steps

Proposition 3.1. *The computational cost of computing $\hat{\theta}$ with the closed formula is $O(nd^2 + d^3)$*

Remark 3.2. *On top of the computational complexity, there is a numerical aspect of the problem that we are ignoring. Namely, inverting ill-conditioned* matrices may yield undesirable results.*

The natural question, as computer scientists, is then whether we can get a "good enough" solution with less computational burden.

4 Gradient Descent

A standard approach in optimization (and in machine learning) is to follow gradient descent! Consider the generic optimization task of minimizing a differentiable function F over $\mathbb{R}^d$, the gradient descent algorithm works as follows:

1. Start with an arbitrary $\theta_0 \in \mathbb{R}^d$
2. For any $t = 1, 2, \ldots,$
   (a) Compute the gradient $\nabla F(\theta_{t-1})$
   (b) Define $\theta_t = \theta_{t-1} - \gamma_t \nabla F(\theta_{t-1})$

*Formally, a matrix is ill-conditioned if the gap between the eigenvalues is large, meaning that there are directions with wildly different scales.

The sequence of γ_t is called the learning rate (or stepsize) of the algorithm. Gradient Descent is typically meaningful when there is no distinction between *local* and *global* minima, see for instance Figure 3. In general, there is indeed always the risk that gradient descent will get trapped in some local minimum, that may be arbitrarily far from the best one. A simple property that avoids this is convexity.

Definition 4.1. *A function $F : \mathbb{R}^d \to \mathbb{R}$ is convex if, for any point x, y and $\alpha \in [0, 1]$, the following inequality holds true:*

$$F(\alpha x + (1 - \alpha)y) \leq \alpha F(x) + (1 - \alpha)F(y).$$

A function F is concave if $-F$ is convex.

An easy corollary of the definition is that for convex functions (defined on the whole $\mathbb{R}^d$) any local minima is also a global one! Furthermore, there is a simple analytical way of assessing whether a function is convex, by just looking at its Hessian H. A function in $C^2(\mathbb{R}^d)$ is convex if and only if its Hessian is positive (semi-)definite. Similar results hold for concavity/maximization and negative definiteness.

Definition 4.2. *A symmetric matrix A is definite semi-positive if $v^T Av \geq 0$ for any vector v. If the inequality is strict only for $v = 0$, then A is positive definite. A matrix A is called negative (semi-)definite, if $-A$ if positive (semi-)definite.*

Proposition 4.3. *Let A be a symmetric matrix, and let $\lambda_{\max}$ be the eigenvalue with the largest absolute value. Then for any vector v, it holds $v^T Av \leq |\lambda_{\max}|||v||_2^2$.*

Proof. The matrix is symmetric, so it is possible to diagonalize it. Let $v_1, \ldots, v_n$ be a basis of orthonormal eigenvectors, corresponding to eigenvalues $\lambda_1, \ldots, \lambda_n$. We can rewrite v according to this new basis: $v = \sum_i \alpha_i v_i$. We have the following:

$$\begin{aligned}
v^T Av &= (\sum_i \alpha_i v_i^T) A (\sum_j \alpha_j v_j) \\
&= (\sum_i \alpha_i v_i^T)(\sum_j \alpha_j \lambda_j v_j) && \text{(Definition of eigenvectors)} \\
&= \sum_{i,j} \alpha_i \alpha_j \lambda_j v_i^T v_j \\
&= \sum_i \alpha_i^2 \lambda_i && (v_i \text{ form orthonormal basis}) \\
&\leq \max_i |\lambda_i| \cdot ||v||_2^2.
\end{aligned}$$

Note, in the last inequality, we are using that $||v||_2^2 = \sum_i \alpha_i^2$. □

4.1 Gradient Descent for Linear Regression

Let's see what this algorithm would look like for linear regression, where the objective function we are trying to minimize is $\hat{R}(\theta) = \frac{1}{2n}||\Phi\theta - y||_2^2$. We already know that $\nabla \hat{R} = \frac{1}{n}(\Phi^T \Phi\theta - \Phi^T y)$, while the hessian is $H = \frac{1}{n}\Phi^T \Phi$. It is a simple exercise to show that $\hat{R}$ is convex, and thus gradient descent would converge to the global minimum (if the step-size is appropriate).

Proposition 4.4. *The Hessian matrix $H = \frac{1}{n}\Phi^T \Phi$ is symmetric, and its eigenvalues are positive $0 < \mu = \lambda_1 \leq \cdots \leq \lambda_d = L$. Therefore $\hat{R}$ is convex.*

Proof. We study the matrix $\Phi^T\Phi$, it is symmetric as the generic entry is $(\Phi^T\Phi)_{i,j} = \phi_i^T(x)\phi_j(x) = \sum_{\ell=1}^n \phi_i(x_\ell)\phi_j(x_\ell) = \phi_j^T(x)\phi_i(x) = (\Phi^T\Phi)_{j,i}$. Symmetry implies diagonalizability, moreover for any eigenvector v and corresponding eigenvalue λ it holds:

$$(\Phi^T\Phi)v = \lambda v \implies ||\Phi v||_2^2 = v^T\Phi^T\Phi v = \lambda||v||_2^2 \implies \lambda = \frac{||\Phi v||_2^2}{||v||_2^2} > 0.$$

In the above inequality, we prove that $v^THv \geq 0$ for any $v \neq 0$, meaning that H is positive definite; this implies convexity. □

The gradient descent algorithm for linear regression thus works as follows:

1. Set the initial θ_0 in an arbitrary way
2. For $t = 1, \ldots, N$,
   - Compute $\theta_t = \theta_{t-1} - \gamma_t \nabla F(\theta_{t-1}) = \theta_{t-1} - \gamma_t \frac{1}{n}(\Phi^T\Phi\theta_{t-1} - \Phi^T y)$

We massage the formula for the generic term, setting $\gamma_t = \gamma$ for simplicity:

$$\begin{aligned}
\theta_t - \hat{\theta} &= \theta_{t-1} - \gamma\frac{1}{n}(\Phi^T\Phi\theta_{t-1} - \Phi^T y) - \hat{\theta} \\
&= (\mathbf{I} - \gamma H)\theta_{t-1} + \gamma\frac{1}{n}\Phi^T y - \hat{\theta} \\
&= (\mathbf{I} - \gamma H)(\theta_{t-1} - \hat{\theta}) && \text{(By definition } \Phi^T y = (\Phi^T\Phi)\hat{\theta}) \\
&= (\mathbf{I} - \gamma H)^2(\theta_{t-2} - \hat{\theta}) \\
&= (\mathbf{I} - \gamma H)^t(\theta_0 - \hat{\theta}).
\end{aligned}$$

If we now look at the distances, we get:

$$\begin{aligned}
||\hat{\theta}_t - \hat{\theta}||_2^2 &= (\theta_0 - \hat{\theta})^T(\mathbf{I} - \gamma H)^{2t}(\theta_0 - \hat{\theta}) && \text{((}\mathbf{I} - \gamma H\text{) is symmetric)} \\
\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) &= \frac{1}{2}(\theta_0 - \hat{\theta})^T(\mathbf{I} - \gamma H)^t H(\mathbf{I} - \gamma H)^t(\theta_0 - \hat{\theta}) \\
&= \frac{1}{2}(\theta_0 - \hat{\theta})^T(\mathbf{I} - \gamma H)^{2t} H(\theta_0 - \hat{\theta}),
\end{aligned}$$

where the last equality follows by observing that H commutes with both $\mathbf{I}$ and γH. We now have all the ingredients to formally study the convergence rate of gradient descent. To this end, recall that μ and L are the smallest, respectively, largest eigenvalues of H. Denote with $\kappa = {}^L/_\mu \geq 1$ the *condition number*.

Theorem 4.5. *Consider gradient descent for linear regression with learning rate $\gamma_t = {}^1/_L$. The following bounds hold:*

$$\begin{aligned}
||\theta_t - \hat{\theta}||_2^2 &\leq \left(1 - \frac{1}{\kappa}\right)^{2t} ||\theta_0 - \hat{\theta}||_2^2 && \text{(Convergence in } \theta\text{)} \\
\hat{R}(\theta_t) - \hat{\mathbb{R}}(\hat{\theta}) &\leq \left(1 - \frac{1}{\kappa}\right)^{2t} \hat{R}(\theta_0) - \hat{\mathbb{R}}(\hat{\theta}) && \text{(Convergence in value of } \hat{R}\text{)} \\
\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) &\leq \frac{L}{4t}||\theta_0 - \hat{\theta}||_2^2 && \text{(Convergence in value w.r.t. initial } \theta \text{ error)}
\end{aligned}$$

Proof. We know that $||\hat{\theta}_t - \hat{\theta}||_2^2 = (\theta_0 - \hat{\theta})^T(\mathbf{I} - \gamma H)^{2t}(\theta_0 - \hat{\theta})$, so Proposition 4.3 tells me that we need to look at the eigenvalues of $\mathbf{I} - \gamma_t H$. Such a matrix is symmetric (sum of two symmetric matrices), and has eigenvalues of the form $(1 - \gamma\lambda_i)$, for λ_i eigenvalue of H. Setting $\gamma = 1/L$ yields that the eigenvalues of $\mathbf{I} - \gamma H$ are of the form $1 - \lambda_i/L$, which is non-negative for any λ_i. Therefore, the matrix is positive semi-definite, and we can apply the Proposition 4.3. We have:

$$||\hat{\theta}_t - \hat{\theta}||_2^2 \leq \max_i \left(1 - \frac{\lambda_i}{L}\right)^{2t} ||\theta_0 - \hat{\theta}||_2^2 = \left(1 - \frac{1}{\kappa}\right)^{2t} ||\theta_0 - \hat{\theta}||_2^2. \tag{1}$$

For the third inequality in the statement, we need to look at $H(\mathbf{I} - \gamma_t H)^2 t$. We bound the corresponding eigenvalues $\lambda_i(1 - \lambda_i/L)^{2t}$:

$$\begin{aligned}
\lambda\,(1 - \gamma\lambda)^{2t} &\leq \lambda e^{-2t\gamma\lambda} && \text{(Because } (1 - \tfrac{1}{x})^x \leq e^{-1} \text{ for any } x > 1\text{)} \\
&\leq \frac{2t\gamma\lambda}{2t\gamma} e^{-2t\gamma\lambda} \leq \frac{1}{2t\gamma} \sup_{\alpha \geq 0} \alpha e^{-\alpha} \\
&\leq \frac{1}{2et\gamma} \leq \frac{1}{4t\gamma} && \text{(Because } \sup_{\alpha\geq 0} \alpha e^{-\alpha} = e^{-1}\text{)}
\end{aligned}$$

Setting $\gamma = 1/L$ yields the desired result. We are left with arguing about the second inequality, which entails bounding the difference $\hat{R}(\theta_t) - \hat{R}(\hat{\theta})$ in terms of the initial error $\hat{R}(\theta_0) - \hat{R}(\hat{\theta})$.

Let $v_1, \ldots, v_d$ be an orthonormal basis of $\mathbb{R}^d$, composed by eigenvectors of H (so that $Hv_i = \lambda_i v_i$), and denote with $(\alpha_1, \ldots, \alpha_d)$ the coordinates of $(\theta_0 - \hat{\theta})$ with respect to such basis.

We have the following chain of inequalities:

$$\begin{aligned}
\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) &= \frac{1}{2}(\theta_0 - \hat{\theta})^T(\mathbf{I} - \gamma H)^{2t} H(\theta_0 - \hat{\theta}) && \text{(Proved above)} \\
&= \left(\sum_i \alpha_i v_i^T\right)(\mathbf{I} - \gamma H)^{2t} H \left(\sum_j \alpha_i v_j\right) && \text{(By definition of the coefficients } \alpha_i\text{)} \\
&= \left(\sum_i \alpha_i v_i^T\right) \sum_j (1 - \gamma\lambda_j)^{2t} \lambda_j \alpha_j v_j && (v_j \text{ eigenvec. of } H \text{ and thus of } \mathbf{I} - \gamma H) \\
&= \sum_{i,j} \alpha_i (1 - \gamma\lambda_j)^{2t} \lambda_j \alpha_j v_i^T v_j && \text{(By linearity)} \\
&= \sum_j \alpha_j^2 (1 - \gamma\lambda_j)^{2t} \lambda_j && (v_i^T v_j \text{ is 1 if } i = j \text{ and 0 otherwise}) \\
&\leq \left(1 - \frac{1}{\kappa}\right)^{2t} \sum_j \alpha_j^2 \lambda_j && \text{(Same calculations as in Equation (1) and } \lambda_j \geq 0\text{)} \\
&= \left(1 - \frac{1}{\kappa}\right)^{2t} (\theta_0 - \hat{\theta})^T H(\theta_0 - \hat{\theta}) && \text{(Def. of eigenvec. and of coordinates } \alpha_i\text{)} \\
&= \left(1 - \frac{1}{\kappa}\right)^{2t} (\hat{R}(\theta_0) - \hat{R}(\hat{\theta})).
\end{aligned}$$

This concludes the proof of the Theorem. □

Consider now the running time of gradient descent:

- Computing $\Phi^T\Phi$ costs $O(nd^2)$ time steps

- Computing $\Phi^T y$ costs $O(nd)$ time steps
- The running time of the generic step is $O(d^2)$
- Overall the running time is $O(nd^2 + d^2 N)$, where N is the number of iterations

Corollary 4.6. *After $N = \frac{\kappa}{2} \log \frac{1}{\varepsilon}$ iterations of gradient descent, it holds that $||\theta_t - \hat{\theta}||_2^2 \leq \varepsilon ||\theta_0 - \hat{\theta}||_2^2$.*

Proof. Consider the generic iteration t, we have the following:

$$\left(1 - \frac{1}{\kappa}\right)^{2t} \leq e^{-\frac{2t}{\kappa}}. \qquad \text{(Because } (1 - \tfrac{1}{x})^x \leq e^{-1} \text{ for any } x > 1)$$

We thus want to find t such that $e^{-\frac{2t}{\kappa}} \leq \varepsilon$, we apply the log to both terms and get the desired bound. □

All in all, we have proved that the computational complexity of decreasing the error by a factor ε is $O(nd^2 + d^2 \kappa \log \frac{1}{\varepsilon})$. Instead of computing $\Phi^T \Phi$ in advance (it takes $O(nd^2)$), one can compute the gradient from scratch at each time step, for a running time of $O(nd)$. With this approach, the bound becomes $O(ndN) = O(\kappa \log \frac{1}{\varepsilon} nd)$.

4.2 Stochastic Gradient Descent

The downside of gradient descent is that a typical iteration of the algorithm entails a pass through the whole dataset. This phenomenon does not only happen for linear regression, but for all loss function of the forms $L(\theta) = \sum_i \ell_i(\theta)$, where ℓ_i is the loss associated to the ith sample[†]. Indeed, the gradient of L is given by the sum of the n gradients of the ℓ_i terms, each depending on one single data point. A natural way of getting around this computational bottleneck is to use a random proxy from ∇L. For instance, we could use the random variable g defined below:

$$g(\theta_t) = \ell_{i_t}(\theta_t), \quad \text{where } i_t \text{ is a point chosen uniformly at random}$$

It is a simple exercise to see that $\mathbb{E}\left[g(\theta_t)\right] = \nabla L(\theta)$, meaning that *in expectation*, the random vector g points in the right direction. To improve the variance of the random variable g, we could use the so-called minibatch approach, namely, instead of drawing one point i_t uniformly at random, we could sample some of them and use them to get a more robust estimate of the gradient in θ_t.

5 Pointers

The discussion on Polynomial Regression and overfitting is inspired from Chapter 1.1 of Bishop [2007], while the sections on gradient and stochastic gradient descents follow the presentation of Bach [2024]. For a quick reference on linear algebra, we point to Axler [2025].

References

Sheldon Axler. *Linear Algebra Done Right*. Springer, 4th edition, 2025. URL `https://linear.axler.net/`.

[†]For linear regression, we have that $\ell_i(\theta)$ represents the squared error from $\phi(x_i)^T \theta$ and y_i, divided by the averaging factor $2n$.

Francis Bach. *Learning Theory from First Principles*. The MIT Press, 2024. URL https://www.di.ens.fr/~fbach/ltfp_book.pdf.

Christopher M. Bishop. *Pattern recognition and machine learning, 5th Edition*. Information science and statistics. Springer, 2007. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf.